

Department of Artificial Intelligence and Machine Learning

Assignment Questions

Course Code & Name : MLA03 – Reinforcement Learning

V. Ganga Deepika

192325072

Assignment Title:	Develop a multi-agent reinforcement learning framework that combines policy-based and model-based RL for traffic optimization. Explain how hierarchical learning and meta-learning can be incorporated to improve coordination, planning, and adaptation to unexpected traffic conditions.
Course Outcome(s) – CO:	CO3 Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. CO4 Evaluate model-based reinforcement learning approaches for prediction, planning, and policy optimization. CO5 Apply hierarchical, meta-learning, and multi-agent RL approaches to solve complex decision-making problems
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 11 – Sustainable Cities and Communities; SDG 13 – Climate Action

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Requirement Understanding, Data Types, Operators & Control Structures (CO1)	10	Correctly identifies all entities and workflow from the problem statement; data types, operators, and control structures used efficiently and correctly throughout.	Identifies most entities/workflow with minor gaps; mostly correct use of data types, operators, and control structures.	Basic entities/workflow identified; data types and control structures used but with some inefficiencies or errors.	Requirements misunderstood; frequent misuse of data types/operators/control flow affecting correctness.
Class Design: Encapsulation, Access Specifiers & Member Functions (CO2)	15	Classes for Vehicle, ChargingPoint, and Session are well encapsulated with appropriate access specifiers; member functions are cohesive and correctly scoped.	Reasonable encapsulation with minor access-specifier misuse.	Classes exist but encapsulation is weak (e.g., excessive public members).	Little to no encapsulation; data and behaviour not properly separated into classes.
Static Members & Arrays of Objects (CO2)	10	Static members correctly used for shared/station-level data; arrays/collections of objects correctly manage multiple vehicles/sessions.	Static members and object arrays used with minor logical errors.	Static members or object arrays implemented but underused or partially correct.	Static members and/or arrays of objects missing or non-functional.

Inheritance Hierarchy & Polymorphism (Virtual Functions) (CO2/CO3)	15	Clear inheritance hierarchy for vehicle/charger types; virtual functions correctly enable run-time adaptation of charging behaviour, tariff computation, and session comparison.	Inheritance and virtual functions are implemented correctly but adaptation logic is limited to one or two behaviours.	Basic inheritance present but shallow; virtual functions declared but overriding are incomplete or inconsistent.	No meaningful inheritance hierarchy or polymorphism implemented.
Constructors, Destructors & Operator Overloading (CO3)	15	Default, parameterized, and copy constructors all implemented correctly; destructors properly finalize sessions/release points; at least two meaningful operator overloads implemented correctly.	Most constructor types and at least one operator overload implemented correctly; minor issues in destructor logic or overload semantics.	Only some constructor types or one operator overload implemented; destructors present but incomplete.	Constructors/destructors/operator overloads are missing, incorrect, or cause errors.
Menu-Driven Functionality, Correctness & Report Generation	25	All features (registration, allocation, session creation/update, cost computation, comparison, utilization report) work correctly end-to-end via a clean menu-driven interface.	Most features work correctly; minor bugs in edge cases or report formatting.	Core features work but some functionality (e.g., report or tariff calculation) is incomplete or buggy.	Program does not run reliably or major features are missing/non-functional.

Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 7/11/12 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; sustainability connection and challenges mentioned but lack depth.	Reflection present but generic; limited justification of design or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no real design/SDG/learning connection.
---	----	--	---	--	---

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

Multi-Agent Reinforcement Learning Framework for Traffic Optimization

Problem Statement

Develop a Multi-Agent Reinforcement Learning (MARL) framework for intelligent traffic optimization in which each traffic intersection is represented as an independent agent. The framework should combine policy-based reinforcement learning for selecting traffic-signal actions and model-based reinforcement learning for predicting future traffic conditions and supporting planning. Hierarchical Reinforcement Learning (HRL) should be incorporated to coordinate traffic at high and low levels, while meta-learning should enable agents to quickly adapt to unexpected conditions such as accidents, road closures, sudden traffic increases, and changing traffic patterns. The system should allow agents to communicate and cooperate with neighboring intersections to reduce congestion, waiting time, and queue length while improving traffic throughput.

Objective

The main objectives of the proposed system are:

- To develop a multi-agent RL framework for traffic signal optimization.
- To use **policy-based RL** for selecting appropriate traffic-signal actions.
- To use **model-based RL** for predicting future traffic conditions.
- To incorporate **hierarchical RL** for high-level and low-level traffic control.
- To use **meta-learning** for rapid adaptation to unexpected traffic conditions.
- To enable communication and coordination between neighboring traffic agents.
- To reduce vehicle waiting time and queue length.
- To minimize traffic congestion.
- To improve traffic throughput and overall traffic flow.

Requirements and Environment Used

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Intel i3 processor or above
- Internet connection for installing required packages

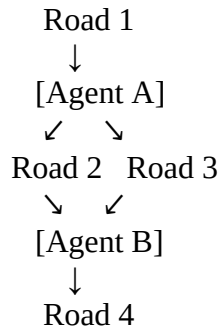
Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab / VS Code
- NumPy
- Matplotlib
- Random library

Environment

A simulated traffic environment is used where multiple intersections are represented by different agents. Each agent observes traffic conditions, selects an action, receives a reward, and updates its decision-making strategy.

Example:



Each agent controls one traffic intersection and communicates with neighboring agents.

Design / Proposed Solution

The proposed framework consists of several components.

Traffic State

Each agent observes:

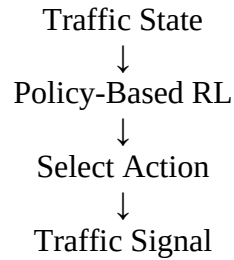
$$S = [Q, D, W, P, V]$$

Where:

- **Q** = Queue length
- **D** = Vehicle density
- **W** = Waiting time
- **P** = Current signal phase
- **V** = Average vehicle speed

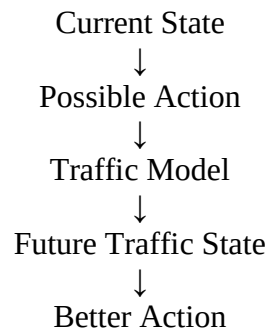
Policy-Based RL

A policy-based algorithm such as **PPO** determines the best action for the current traffic state.



Model-Based RL

The model predicts what may happen after an action is taken.



Thus, policy-based RL provides decision-making while model-based RL provides prediction and planning.

Hierarchical RL

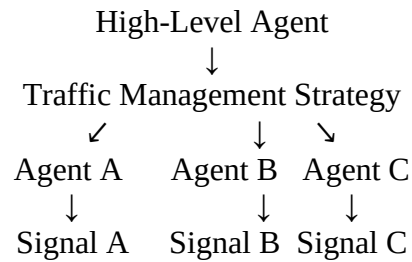
The traffic system is divided into two levels.

High-Level Controller

- Controls overall traffic strategy.
- Identifies congested areas.
- Coordinates multiple intersections.

Low-Level Agents

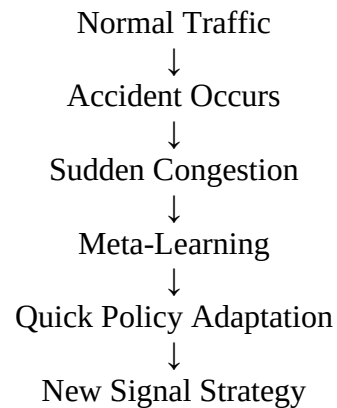
- Control individual traffic signals.
- Adjust green-light duration.
- Change signal phases.



Meta-Learning

Meta-learning helps the agents adapt quickly when traffic conditions change.

For example:



This avoids requiring the agent to learn the completely new situation from the beginning.

Pseudo Code

BEGIN

Initialize traffic environment

Create multiple traffic agents

FOR each time step:

 Observe traffic state

 FOR each agent:

 Select action using Policy-Based RL

Predict future traffic using Model-Based RL

Coordinate with neighboring agents

Apply traffic signal action

Calculate reward

Update policy

IF unexpected traffic occurs:

 Apply Meta-Learning

 Adapt policy quickly

Use Hierarchical RL to coordinate

high-level and low-level decisions

END FOR

Return optimized traffic policy

END

Implementation / Source Code

```
import random
# Number of traffic intersections
num_agents = 3
# Initial traffic state for each agent
traffic = [
    [20, 0.7, 30], # Agent 1: queue, density, waiting time
    [10, 0.4, 15], # Agent 2
    [25, 0.8, 40]  # Agent 3
]
```

```

# Possible actions
actions = ["Keep Green", "Extend Green", "Change Signal"]

# Policy-Based RL
def policy_action(state):
    queue, density, waiting = state
    if queue > 20 or waiting > 30:
        return "Extend Green"
    elif density > 0.6:
        return "Keep Green"
    else:
        return "Change Signal"

# Model-Based RL
def predict_future(state, action):
    queue, density, waiting = state
    if action == "Extend Green":
        predicted_queue = max(0, queue - 5)
    elif action == "Change Signal":
        predicted_queue = queue + 3
    else:
        predicted_queue = queue
    return predicted_queue

# Reward Function
def calculate_reward(queue, waiting):
    return -(queue + waiting)

# Hierarchical RL
def high_level_controller(traffic):
    total_queue = sum(state[0] for state in traffic)
    if total_queue > 50:
        return "High Congestion"
    else:
        return "Normal Traffic"

```

```

# Meta-Learning
def meta_learning(condition):
    if condition == "Accident":
        print("Unexpected traffic detected!")
        print("Meta-learning: Adapting policy quickly")
# Main Program
for step in range(5):
    print("\nTime Step:", step + 1)
    # High-level decision
    strategy = high_level_controller(traffic)
    print("High-Level Strategy:", strategy)
    for i in range(num_agents):
        print("\nAgent", i + 1)
        # Observe state
        state = traffic[i]
        print("State:", state)
        # Policy-based RL selects action
        action = policy_action(state)
        print("Policy Action:", action)
        # Model-based RL predicts future
        predicted_queue = predict_future(state, action)
        print("Predicted Queue:", predicted_queue)
        # Apply action
        if action == "Extend Green":
            traffic[i][0] = max(0, traffic[i][0] - 5)
            traffic[i][2] = max(0, traffic[i][2] - 5)
        elif action == "Change Signal":
            traffic[i][0] += 3
            traffic[i][2] += 2
    # Calculate reward
    reward = calculate_reward(

```

```

        traffic[i][0],
        traffic[i][2]
    )
    print("Reward:", reward)
# Simulate unexpected traffic
if step == 2:
    meta_learning("Accident")
print("\nFinal Traffic State:")
for i in range(num_agents):
    print("Agent", i + 1, ":", traffic[i])

```

Output

```

***
Time Step: 1
High-Level Strategy: High Congestion

Agent 1
State: [20, 0.7, 30]
Policy Action: Keep Green
Predicted Queue: 20
Reward: -50

Agent 2
State: [10, 0.4, 15]
Policy Action: Change Signal
Predicted Queue: 13
Reward: -30

Agent 3
State: [25, 0.8, 40]
Policy Action: Extend Green
Predicted Queue: 20
Reward: -55

Time Step: 2
High-Level Strategy: High Congestion

Agent 1
State: [20, 0.7, 30]
Policy Action: Keep Green
Predicted Queue: 20
Reward: -50

Agent 2
State: [13, 0.4, 17]
Policy Action: Change Signal
Predicted Queue: 16
Reward: -35

Agent 3
State: [20, 0.8, 35]
Policy Action: Extend Green
Predicted Queue: 15
Reward: -45

```

```

*** Time Step: 3
High-Level Strategy: High Congestion

Agent 1
State: [20, 0.7, 30]
Policy Action: Keep Green
Predicted Queue: 20
Reward: -50

Agent 2
State: [16, 0.4, 19]
Policy Action: Change Signal
Predicted Queue: 19
Reward: -40

Agent 3
State: [15, 0.8, 30]
Policy Action: Keep Green
Predicted Queue: 15
Reward: -45
Unexpected traffic detected!
Meta-learning: Adapting policy quickly

Time Step: 4
High-Level Strategy: High Congestion

Agent 1
State: [20, 0.7, 30]
Policy Action: Keep Green
Predicted Queue: 20
Reward: -50

Agent 2
State: [19, 0.4, 21]
Policy Action: Change Signal
Predicted Queue: 22
Reward: -45

Agent 3
State: [15, 0.8, 30]
Policy Action: Keep Green
Predicted Queue: 15
Reward: -45

```

```

Time Step: 5
High-Level Strategy: High Congestion

Agent 1
State: [20, 0.7, 30]
Policy Action: Keep Green
Predicted Queue: 20
Reward: -50

Agent 2
State: [22, 0.4, 23]
Policy Action: Extend Green
Predicted Queue: 17
Reward: -35

Agent 3
State: [15, 0.8, 30]
Policy Action: Keep Green
Predicted Queue: 15
Reward: -45

Final Traffic State:
Agent 1 : [20, 0.7, 30]
Agent 2 : [17, 0.4, 18]
Agent 3 : [15, 0.8, 30]

```

Analysis and Discussion

The proposed framework combines different reinforcement learning techniques to improve traffic management. Policy-based RL helps agents select appropriate signal actions based on current

traffic conditions. Model-based RL provides future traffic predictions, allowing the system to make better planning decisions. Hierarchical RL reduces the complexity of controlling a large number of intersections by dividing the problem into high-level and low-level decisions.

Multi-agent coordination allows neighboring intersections to exchange traffic information and cooperate instead of making independent decisions. Meta-learning is particularly useful when unexpected events occur because the agents can adapt their policies quickly. The major performance measures are average waiting time, average queue length, average travel time, traffic throughput, and cumulative reward.

Overall, the combination provides a scalable approach for intelligent traffic control and can help reduce congestion while improving traffic flow.

Conclusion

The proposed Multi-Agent Reinforcement Learning framework provides an intelligent solution for traffic optimization. Policy-based RL is used for action selection, while model-based RL supports prediction and planning. Hierarchical RL manages traffic at different control levels, and meta-learning enables rapid adaptation to unexpected traffic conditions. Communication between multiple agents improves coordination among intersections. The proposed system can reduce waiting time, queue length, and congestion while improving traffic throughput and adaptability.

References

1. Sutton, R. S., & Barto, A. G., Reinforcement Learning: An Introduction, MIT Press.
2. Busoniu, L., Babuska, R., & De Schutter, B. — *A Comprehensive Survey of Multiagent Reinforcement Learning*, IEEE Transactions on Systems, Man, and Cybernetics, Part C.
3. Vinyals, O., et al. — *Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning*, Nature, 2019.
4. Bacon, P.-L., Harb, J., & Precup, D. — *The Option-Critic Architecture*, AAAI, 2017.
5. Finn, C., Abbeel, P., & Levine, S. — *Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks*, ICML, 2017.

Test Cases and Expected/Actual Results

Test Case	Input Condition	Expected Result	Actual Result
Test Case 1	Low traffic	Change/normal signal control	Agent changes signal
Test Case 2	High queue length	Extend green time	Green time is extended
Test Case 3	High vehicle density	Maintain suitable green phase	Agent keeps green
Test Case 4	High congestion	High-level controller detects congestion	High congestion detected
Test Case 5	Accident	Quickly adapt signal strategy	Meta-learning adaptation activated